

技术文档 · TECHNICAL DOCUMENT

Haven-Store

二手商品交易平台的 AI 辅助开发技术文档：系统阐述本次全方位升级的意图控制策略（如何设计上下文与 Prompt）、架构设计（AI 生成代码采用的架构与约束手段）与可复现性论证。升级覆盖 47 项经人工审查的清单条目，交付 336 项冒烟断言与 66 项单元测试全绿的可复现基线。

版本 1.0 · 2026 年 10 月

目录

章节	内容
第一章 项目概述	本项目定位、升级范围与技术栈
第二章 意图控制策略	人机协作流程、上下文设计、Prompt 设计原则与防跑偏机制
第三章 架构设计	总体架构、后端分层、鉴权链路、并发一致性、前端架构与对 AI 代码的约束手段
第四章 可复现性论证	复现性声明、依赖锁定、配置即代码、行为断言与非确定性分析
第五章 总结与展望	成效总结与后续路线
附录 A 环境变量表	全部可注入配置项
附录 B 接口清单摘要	按模块归纳的 REST 端点
附录 C 常见问题	复现时的典型故障与处置

第一章 项目概述

1.1 项目简介与升级范围

Haven-Store 是一个基于 Spring Boot 与 Vue 3 的全栈二手商品交易平台，支持商品发布、附近商品检索、AI 智能估价、商品评价、收藏与购物车、下单支付（模拟支付）、私信与商品咨询、系统消息、管理后台等能力。本轮工作对项目进行了一次全方位升级：先由 AI 代理对前后端做全量只读盘点，产出结构化的优化清单并经人工审查，随后按审查通过的清单分五个阶段实施，最终以 336 项冒烟断言与 66 项单元测试全部通过作为完成标志。

升级范围覆盖五类工作：必修缺陷修复（10 项，含 1 个权限漏洞与 1 个假功能修复）、普通用户端业务补全（12 项）、管理员端业务补全（9 项）、前端体验优化（10 项）以及安全与工程化加固（10 项）。本文档重点阐述其中的三个横切主题：意图控制策略、架构设计、可复现性论证。

1.2 技术栈

层	技术选型	版本	说明
后端框架	Spring Boot	2.7.12	Tomcat 内嵌，统一上下文路径 /api
ORM	MyBatis-Plus	3.5.3.1	分页插件 maxLimit=200, LambdaQueryWrapper 参数化查询
鉴权	JWT (HS256)	0.11.5	拦截器级登录态解析 + 账号状态校验
存储	MySQL 8 / Redis 7	—	业务数据与会话缓存、登录锁定计数、系统设置
文档	Knife4j (Swagger2)	4.1.0	接口文档，生产 profile 下关闭
前端框架	Vue 3 + Vite 5	^3.4	Composition API，按路由模块分包
UI 与状态	Element Plus / Pinia	^2.4 / ^2.1	中文语言包，全站组件库

层	技术选型	版本	说明
可视化	ECharts	^5.5	管理看板趋势图与分类分布图，独立分包

1.3 交付成果概览

类别	数量	代表性成果
必修缺陷	10 项	注册功能打通；系统消息伪造漏洞摘除；禁用账号 Token 立即失效；分类排序假保存、设置页无法保存等"假功能"修复
用户端补全	12 项	真首页、通知中心、订单详情页、我的发布管理页、AI 估价入口、附近商品、评价与订单挂钩、商品举报、搜索排序增强、订单超时自动取消
管理端补全	9 项	订单管理（含导出）、评论管理、举报处理台、用户编辑与重置密码、ECharts 数据看板、系统设置落库并真实生效、群发消息批次化
体验优化	10 项	共享组件（ApplImage/ProductCard）、全局进度条、favicon、移动端适配收口、死代码清理
安全工程	10 项	登录防爆破、上传魔数校验、Redis 序列化白名单、CSV 公式注入防护、ESLint/Prettier 补齐、测试全绿

1.4 升级前后对比

维度	升级前	升级后
安全	/message/send 任何登录用户可伪造公告；禁用账号 Token 7 天内仍可交易	漏洞端点摘除；拦截器级账号状态校验，禁用即时失效；登录锁定与密码策略生效
用户端	注册页是空壳；首页无商品内容；无通知/订单详情/我的发布页	全链路可用：注册-发布-搜索-附近-估价-下单-支付-评价-举报-私信-通知
管理端	看不到订单与评论；无举报处理；系统设置是摆设	订单/评论/举报/用户增强/看板图表齐备；设置落库并真实约束业务
测试	冒烟 26 项失败；单测 66 项中 63 项从未通过	冒烟 336 项全绿；单测 66 项全绿
工程	无 ESLint 配置、图标库幽灵依赖、多处死代码	配置补齐、依赖显式声明、死代码清零、echarts 独立分包

第二章 意图控制策略

AI 辅助开发最大的风险不是"写不出代码", 而是"写出的代码与意图偏离": 自作主张扩大范围、引入未审查的方案、或者凭想象编造不存在的接口。本章阐述本项目在升级过程中如何通过流程设计、上下文工程与 Prompt 约束, 把大型改造任务的风险控制在可审查、可验证的轨道上。

2.1 总体思路：先对齐、再动手、逐步验证

整个升级遵循一条固定的协作流水线, "生成代码"只是其中占时最短的一环:

- ① 全量盘点 (两个只读探索代理并行扫描前后端)
 - ② 产出结构化优化清单 (按 A 必修缺陷 / B 用户端 / C 管理端 / D 体验 / E 安全 / F 工程化 分组编号)
 - ③ 人工审查清单 (用户逐条确认, 可选项显式二选一, 如 C8 审核流选"轻量方案")
 - ④ 按审查结论排定实施顺序 (A → E 先行 → B → C → D/F), 建立贯穿全程的任务清单
 - ⑤ 分阶段实施, 每阶段以"编译通过 + 冒烟测试"作为完成定义
 - ⑥ 全量回归 (单元测试 66 项 + 冒烟测试 336 项 + 前端构建)

关键在于: 意图不是一句"全方位升级", 而是被拆解成 47 项带编号、带优先级、带验收说明的具体条目; AI 不允许绕过清单自由发挥, 每一处改动都能追溯回某个编号。

2.2 上下文设计

2.2.1 结构化盘点先行，而非边写边猜

在写第一行代码之前, 两个只读探索代理以"very thorough"力度分别扫描后端 (约 50 个端点) 与前端 (20 个视图), 产出包含接口清单、业务现状、安全与健壮性问题清单 (带文件行号)、配置审查、缺失功能对照表在内的结构化报告。这份报告是最重要的"上下文投资": 后续每一次代码生成都在这份共同事实的基础上进行, 避免了模型在缺乏全局视野时编造接口或误判现状 (例如正是盘点发现前端 user.js 中封装了 10 个后端根本不存在的接口)。

2.2.2 上下文分级加载

层级	内容	加载时机	示例
需求级	用户审查通过的 47 项优化清单与裁剪决策	全程常驻	C8 选轻量审核流、本轮不做 WebSocket
架构级	技术栈、分层约定、统一返回结构、鉴权链路	进入某模块前	Result{code,msg,data} 契约、域分包模式
局部级	待修改文件的完整内容与关联文件	每次编辑前	改 ProductServiceImpl 前先读全部 900 行
事实级	数据库 Schema、枚举取值、接口路径	涉及契约时	product.status 仅 1/2/3, 订单状态仅 1-4

分级的意义在于控制上下文窗口的信噪比：局部级信息（如一个 900 行的服务类）只在编辑该文件时读入，读完即用；而需求级信息（用户裁剪决策）永远在场，保证任何时刻的生成都不会越过既定范围。

2.2.3 任务清单作为持久上下文

全程维护一份与清单编号对应的任务清单（todo list），每完成一项立即勾选。它的作用不只是进度展示：当会话上下文因长度发生摘要压缩时，任务清单是唯一跨越压缩边界保留的"总纲"，确保恢复后的生成仍然锚定在审查过的范围上。

2.3 Prompt 设计原则

原则	做法	反例（被禁止的做法）
验收标准前置	每阶段开始前先声明"完成的定义"：编译零错误、冒烟全绿、关键页面可走查	先写代码再想怎么验证
约束优先于指令	显式列出"本轮不做"：真实支付网关、WebSocket 推送、Spring Boot 3 迁移、暗色模式，并写明理由	只说"做什么"，让 AI 自行决定边界
对齐既有契约	新代码的字段名、枚举值、状态码必须引用既有常量（OrderStatusEnum、ProductStatusEnum、ResultCodeEnum），禁止魔法值	在前后端各写一套硬编码状态码
决策显式化	可选项要求"二选一并说明理由"，结论回写进清单与文档	让 AI 在多个方案间悄悄折中
小步提交	每个阶段结束时给出可验证节点，而非憋一个最终大交付	一次性生成全部改动再统一验证

其中"对齐既有契约"在实践中被证明最有价值：本轮发现的 26 项冒烟测试失败中，多数正是新旧代码契约错位（例如设置保存后 maxPrice=100 真实生效导致发布被拒、密码策略 medium 拒绝纯数字演示口令）。这些失败本身就是契约生效的证据，修正方向是让测试与文档对齐新契约，而不是放宽校验。

2.4 防跑偏与纠偏机制

- 范围闸门：任何清单之外的功能想法一律记入"本轮不做"，不在实施中顺手实现；
- 验证闸门：五个阶段各自以编译与测试收口，第 2 阶段结束后即做了一次全量冒烟，而非等到最后才验证；
- 独立安全钩子：仓库接入的独立安全工具（Mimosa）在写入源码时做实时检查，曾拦截冒烟测试中的硬编码口令写法，迫使其改为环境变量模板（口令来自公开种子数据，属演示凭据）；
- 自省式重写：生成过程中若发现自己写出了重复包装方法或引用了已删除字段，立即停止追加、回读文件并整体重写该段，而不是叠加补丁；
- 反证式确认：对"单测失败是否为本次升级引入"这类关键判断，用 git worktree 在未改动的 HEAD 上复跑同版本测试，取得对照证据后再下结论。

2.5 Prompt 模式示例

以下是从本次升级中提炼的三类高频 Prompt 骨架。共同特征是：把"事实、约束、验收标准"写进指令，把"方案选择权"留给受约束的生成，并把"完成定义"写成可机器判定的条件。

【模式一 · 盘点类】

对 <目录> 做 very thorough 的只读盘点。输出结构化报告，包含：

- 1) 接口清单（方法/路径/功能/权限）；2) 各域业务现状；
- 3) 安全与健壮性问题清单（附文件:行号）；4) 配置审查；
- 5) 对照完整平台列出缺失功能。报告将用于编写升级清单，不要给泛泛建议。

【模式二 · 实施类】

按清单编号 A2 修复：<问题描述>。约束：不引入新依赖；复用既有校验模式；

修改前先读 <文件> 全文。完成定义：mvn compile 零错误，冒烟测试不出现新失败。

【模式三 · 纠偏类】

冒烟测试出现 N 项失败。先分类：属于"新契约生效导致旧断言过时"的，

修正断言并在注释中写明契约依据；属于"代码缺陷"的，修复代码。逐项给出根因。

模式三尤其重要：它把"测试失败"从噪声转化为契约演进的驱动力——本轮 26 项失败全部按此流程归因，其中多数归为"设置与密码策略真实生效导致旧断言过时"，其余为注册手机号生成含字母等测试侧缺陷。

第三章 架构设计

3.1 总体架构

系统为经典前后端分离结构：Vue 3 单页应用经 Vite 代理（生产经 Nginx）访问 Spring Boot 单体服务，数据落在 MySQL 与 Redis。单体内部按业务域垂直切分模块，每个域内再做控制器/服务/持久化分层，横切能力（鉴权、异常、缓存、自动填充、定时任务）收敛到独立的 config 与 task 包。

前端 Vue 3 SPA (views / components / store / router / api 封装)

| HTTP (axios 拦截器统一注入 Token、解包 Result、错误提示)



后端 Spring Boot 单体 (/api 上下文)

└─ 拦截器链: JwtInterceptor (登录态+账号状态) → AdminAuthInterceptor (角色复用)

└─ 业务域: user / product / order / cart / message / admin / report / category

└─ 横切层: config (缓存/序列化/CORS) · task (定时任务) · common (Result/异常/常量)



存储 MySQL 8 (12 张表) + Redis 7 (用户缓存、登录锁定、角标计数)

3.2 后端分层与模块化

模块	职责	本轮主要变更
user	注册/登录/资料/改密/头像	登录锁定、密码策略、会话时效设置接入、清空字段语义
product	商品发布/编辑/上下架/搜索/附近/评论	重新上架与彻底删除、编辑状态校验、事务补齐、评论购买校验、搜索多字段、估价开关
order / cart	下单锁定、状态机、购物车收藏互转	支付/确认条件更新、超时自动取消、卖家可见详情
message	系统消息收件箱、私信与咨询	收件箱分页化、群发批次号、批量插入
admin	后台六页 + 订单/评论/举报/趋势	接口大幅扩展，删除复用商品模块级联逻辑
report (新)	举报提交与处理	全新模块：实体/Mapper/服务/控制器
task (新)	定时维护	超时订单取消、商品到期自动下架
config / utils	缓存、序列化、JWT、文件存储、上下文	缓存管理器显式化、序列化白名单、魔数校验、UserHolder 扩展

3.3 统一契约层

- 统一返回：所有接口返回 `Result{code,msg,data}`，业务码集中在 `ResultCodeEnum`，禁止任何模块自定义魔法值；
- 统一异常：业务错误抛 `BusinessException`，全局处理器兜底为固定文案，堆栈只进日志不外泄；
- 统一校验：入参 DTO 用 JSR-303 注解声明约束（`@Pattern/@Size/@NotNull`），控制器 `@Validated` 生效，服务层只做跨字段与状态类校验；
- 统一填充：`create_time/update_time` 由 `MyMetaObjectHandler` 自动填充，业务代码不再手工赋值。

3.4 鉴权与安全链路

本轮把"登录态解析"与"账号状态校验"合并进同一条拦截器链，消除了"禁用用户 7 天内仍可交易"的窗口期，同时让管理端角色校验零额外查库：

请求 → `JwtInterceptor`

└─ 解析 JWT（签名/过期校验，公开路径可选鉴权）

└─ `getAuthUser(userId)`：Redis `user:info:{id}` 优先，未命中回源 MySQL 并重建缓存（密码不落缓存）

└─ 账号不存在 或 `status=禁用` → 直接写 401 响应（禁用即时失效）

└─ 通过 → `UserHolder` 写入 `userId + User` 实体（请求级复用，`AdminAuth` 不再查库）

→ `AdminAuthInterceptor`（仅 `/admin/**`）：复用 `UserHolder` 中的实体校验 `role=1`

→ 业务 `Controller`：只关心业务规则，权限判断不侵入业务代码

3.5 并发与一致性设计

场景	机制	效果
下单锁商品	UPDATE product SET status=2 WHERE id=? AND status=1 条件更新，影响行数为 0 即回滚	同一件二手商品不会被两人同时买走
支付/确认收货	同款条件更新（WHERE status=期望值），失败提示刷新重试	并发下订单状态不会重复推进、不会跳状态
超时未支付	定时任务逐单条件更新取消并释放商品，与买家手动取消互不冲突	30 分钟未支付的订单确定性回收
系统设置	落库 system_setting 单行表 + 整体覆盖保存，读取失败回落默认值	设置真实约束业务（上传/限价/锁定/时效），且 Redis 重启不丢
分类缓存	@Cacheable 写 Redis（TTL 10 分钟），管理端增删改整组驱逐	高频下拉查询不打库，变更即时可见
上传安全	扩展名白名单（后台可配）+ 文件头魔数校验 + UUID 落盘	伪装成图片的脚本内容被拒绝

3.6 前端架构

前端按"布局 - 视图 - 共享组件 - 基础设施"四层组织。三个布局（MainLayout/AuthLayout/AdminLayout）承载导航与角标；视图按路由模块懒加载；本轮新增的共享组件 ProductCard 与 ApplImage 消除了此前商品卡片在两处页面重复手写、图片坏链无兜底的问题。

- 状态管理：Pinia user store 是唯一状态中心（登录态归一化、未读私信/通知/购物车三个角标），Token 的持久化读写收敛在 utils/auth；
- 请求层：axios 拦截器统一注入 Token、解包 Result、401 清态跳登录、业务错误统一弹提示，页面 catch 中不再重复弹错；
- 契约对齐：utils/constants.js 集中维护与后端一致的枚举映射（订单 1-4、商品 1-3），本轮修正了过时的"7 状态"订单枚举并新增状态映射表供各页复用；
- 移动端：前台顶栏与后台侧边栏补齐汉堡菜单，私信中心双栏在窄屏改上下堆叠。

3.7 数据模型

数据库共 12 张表（本轮新增 2 张、扩展 1 列），核心关系围绕"用户-商品-交易-消息"四条主线展开：

表	职责	关键设计
user	账号	BCrypt 密文、role 区分管理员、status 支持禁用
category / product / product_image	分类与商品	商品状态机 1在售/2已售出/3已下架；图片表首图为封面冗余

表	职责	关键设计
user_product_relation	收藏与购物车	同表不同 relation_type，唯一索引天然防重，互转零成本
product_order / order_item	交易	明细存下单快照（标题/封面/价格），商品后续变更不影响历史订单
comment	评价	status 隐藏位；评分与内容受 DTO 校验
private_message	私信与咨询	会话键由双方用户 ID 排序拼接 + 商品 ID，天然隔离越权
system_message	系统通知	receiver_type 群发标记 + batch_no 批次号（本轮新增）
report（新）	举报	target_type+target_id 多态指向商品或评论，处理动作与留痕
system_setting（新）	系统设置	单行表整体 JSON 覆盖，读取失败回落默认值

3.8 对 AI 生成代码的约束手段

架构约束的本质是"让新代码没有自由发挥的空间"。本项目把约束分成四层，强度递增：

层级	手段	实例
结构约束	沿用既有域分包与分层命名，新模块照抄成熟模板（report 模块完全复刻 message 模块的骨架）	新增 report/task 两个包零架构争议
契约约束	枚举集中、DTO 校验注解、接口注释与 Swagger 注解齐全、Result 统一返回	所有新接口自动被拦截器保护，无一例外
流程约束	编辑前必读文件全文；每阶段编译；关键判断用对照实验取证	避免"盲改"：900 行服务类全读后再动刀
验证约束	冒烟测试把"设置真实生效"、"评论需购买"等新契约固化为断言，回归不通过即不得交付	336 项断言成为架构的行为规范

一个典型的纠偏案例：管理端扩展时曾生成过一组引用已删除字段的包装方法，识别后没有叠加补丁，而是恢复必要的 Mapper 注入并把该段整体重写为直调实现——约束的价值不在于不出错，而在于错误被结构化的流程快速捕获并纠正。

第四章 可复现性论证

4.1 复现性声明

本项目声明：在满足 README 所列环境要求（JDK 8+、Node 18+、MySQL 8、Redis 7）的前提下，任何人从仓库零起步，可在 10 分钟内完成系统复现，并通过 336 项冒烟断言与 66 项单元测试验证行为一致。复现性由四根支柱支撑：环境即代码、配置有默认值可覆盖、行为有断言、无外部付费服务依赖。

4.2 环境与依赖锁定

- 后端依赖由 pom.xml 固定（Spring Boot 2.7.12 parent 统管传递依赖版本），构建产物 second-hand-market-1.0.0.jar；
- 前端依赖由 package.json + package-lock.json 双重锁定，npm install 结果跨机器一致；
- 语言运行时要求宽松（JDK 8+ 即可编译），仓库 .tools/ 内还附带本机专用 Maven 3.9.16，消除工具链差异；
- 唯一外部服务是 MySQL 与 Redis，均为本地部署的常见版本，不依赖任何云服务或付费接口。

4.3 数据与配置即代码

- 建库即重建：backend/scripts/01-schema.sql 一个脚本创建全部 12 张表并写入种子数据（3 个账号、2 件在售商品、8 个分类），不依赖任何手工步骤；
- 增量升级可重放：老库执行 02-upgrade.sql 一次完成本轮的表结构演进（report、system_setting、batch_no）；
- 配置零改动可跑：application.yml 全部环境相关项写成 \${VAR:默认值}，默认值即本地开发可用值，部署时以环境变量覆盖（模板见附录 A）；
- 一键脚本：scripts/ 下提供依赖拉起、建库、重建后端、一键启动四个批处理，复现流程不依赖操作者记忆。

4.4 行为可验证：把复现从"感觉能跑"变成"断言全绿"

复现成功的判定不依赖人工浏览页面，而是两组自动化套件：tests/api_smoke_test.py 以 336 项断言覆盖健康检查、鉴权拦截、管理端全部页面、注册、发布、搜索、上传、评价、私信、购物车、订单全生命周期、举报闭环、数据导出与设置生效；mvn test 的 66 项单元测试覆盖商品服务与 AI 估价服务的边界条件。两组套件都在 1-2 分钟内跑完，任何契约回归都会在分钟级暴露。

冒烟覆盖域	断言内容举例
鉴权与权限	未登录 401、普通用户访问后台 403、无关第三方查看订单 403
交易主链路	下单锁商品、重复支付被拒、取消释放商品、确认收货状态机、订单快照不随商品删除失效

冒烟覆盖域	断言内容举例
设置真实生效	上传类型白名单拒绝 png、价格上限拒绝超限商品、会话超时写入 Token
评价新契约	未购买用户评价被拒、购买后可评、评论带"已验证购买"标识
举报闭环	提交举报、重复举报被拒、管理端按举报下架商品、卖家重新上架
管理端	订单列表/详情/筛选、评论列表、看板趋势与分类分布、导出 CSV

4.5 已知非确定性因素与影响分析

因素	来源	影响范围	控制手段
AI 估价随机波动	估价规则含 ± 200 元扰动，模拟 AI 不确定性	仅估价建议数值，不影响发布	单元测试改为断言区间而非精确值
时间戳	create_time、支付时间等取运行时刻	展示与排序，不影响业务	测试断言不比较绝对时间
并发购买结果	两人同时买同一件商品，胜者取决于竞争顺序	谁下单成功的不确定性	条件更新保证"恰好一人成功"这一确定性
登录锁定计数	失败次数存 Redis，重置环境即清零	锁定时长内的登录尝试	冒烟测试不依赖锁定状态

4.6 复现演练记录

本轮升级完成后进行过一次完整演练，过程与结果如下，可作为复现指南的实测基线：

步骤	内容	结果
环境	Windows 11 / JDK 22 / Node 22 / MySQL 8 / Redis 7（本机服务）	就绪
建库	对既有库执行 02-upgrade.sql，确认 12 张表与 batch_no 列	通过
构建	mvn -DskipTests package 生成 jar；前端 npm run build 通过并完成分包	通过
启动	java -jar 启动后端，/api/health 返回 code=200	通过
单元测试	mvn test: Tests run 66, Failures 0, Errors 0	通过
冒烟测试	python tests/api_smoke_test.py: 通过 336，失败 0	通过
行为抽检	登录 Token 有效期随"会话超时"设置变为 12 小时，验证设置真实生效	通过

第五章 总结与展望

本轮升级验证了一条可复制的 AI 辅助改造路径：以结构化盘点建立共同事实，以经人审查的编号清单锚定意图，以分阶段验证闸门控制风险，最终以自动化断言交付可复现的结果。47 项清单条目全部落地，两套测试从部分失真走向全绿，安全工具与测试套件在过程中各自拦截了真实问题，证明"约束 + 验证"的组合比"生成后人工审读"更可靠。

后续路线建议：接入真实支付网关与 WebSocket 推送（本轮明确延后的两项）、将系统设置中的密码策略与注册风控进一步联动、以及把冒烟测试纳入 CI 在每次提交时自动执行。

附录 A 环境变量表

变量	默认值	说明	部署必改
DB_HOST / DB_PORT	localhost / 3306	MySQL 地址	—
DB_NAME	secondhand_market	库名，与建表脚本一致	—
DB_USERNAME / DB_PASSWORD	root / 1234	数据库凭据	密码必改
REDIS_HOST / REDIS_PORT	localhost / 6379	Redis 地址	—
JWT_SECRET	内置默认串	HS256 签名密钥，≥32 字节	必改
JWT_EXPIRATION	604800000	Token 默认有效期（毫秒），后台安全设置可覆盖	—
FILE_UPLOAD_DIR	./uploads	上传落盘目录	建议挂独立数据卷

附录 B 接口清单摘要

模块	代表性端点	权限
user	POST /user/register · POST /user/login · GET/PUT /user/info · PUT /user/password · POST /user/avatar	注册登录免鉴权，其余需登录
product	POST /product/add · GET /product/search · GET /product/detail/{id} · POST /product/nearby · POST /product/estimate · PUT/DELETE /product/{id} · POST /product/{id}/reshelf · DELETE /product/{id}/purge · GET/POST /product/{id}/comments · POST /product/upload	搜索/详情/分类/评论读公开，其余需登录
report (新)	POST /report	需登录
order	POST /order/create · GET /order/list · GET /order/sold · GET /order/{orderNo} · POST /order/{orderNo}/pay cancel confirm	需登录

模块	代表性端点	权限
cart	POST/DELETE /cart/{productId} · GET /cart/list · GET /cart/count · 购物车收藏互转	需登录
message	GET /message/list · POST /message/{id}/read · POST /message/all/read · GET /message/unread/count · POST /message/private/send · GET /message/private/chat · GET /message/private/conversations	需登录
admin	GET /admin/dashboard · /admin/stats/trend · /admin/stats/category · /admin/orders · /admin/comments · /admin/reports · /admin/users · /admin/products · /admin/categories · /admin/messages · GET/PUT /admin/settings · POST /admin/upload · /admin/export/*	需管理员
public (新)	GET /public/settings	免鉴权

附录 C 常见问题

- 接口全部报 code=501"系统内部错误": MySQL 或 Redis 未启动。运行 scripts/start-dev-services.bat 后无需重启后端, 会自动重连;
- 报 Table secondhand_market.user doesn't exist: 库未初始化, 执行 scripts/init-database.bat 或手动导入 backend/scripts/01-schema.sql;
- 老库启动报 Unknown column batch_no: 执行一次 backend/scripts/02-upgrade.sql;
- 注册报"密码强度不足": 后台安全设置默认 medium (须含字母与数字), 演示可临时调为 low;
- 发布商品报"超过平台上限": 后台交易设置的价格上限真实生效, 按需调整;
- 图片上传失败: 确认上传目录可写、文件为真实图片 (服务端校验文件头), 单张不超过后台设置的大小上限。